Team Flight Engineering Journal

Summer Lu & Nicole Jin

WRO Future Engineers

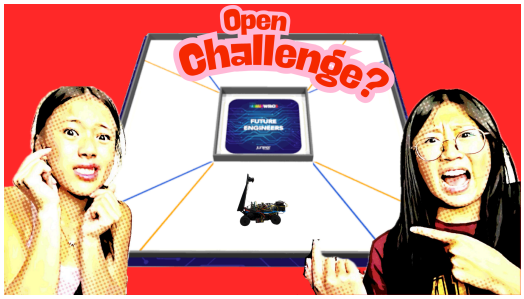
Coach Rice

February 13, 2026

Table of Contents

Team Photo

Links



Open challenge link: [Team Flight Open Challenge 2026]

Obstacle challenge link: [Team Flight Obstacle Challenge 2026]

Introduction

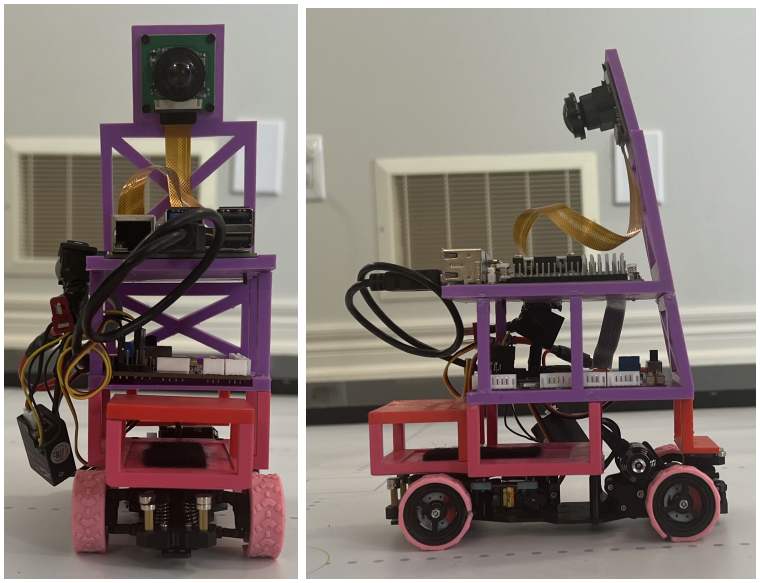
Our vehicle is a lightweight, high-torque autonomous racer built on a custom brushless drivetrain. By integrating a Raspberry Pi 5 for vision and an Arduino Nano 33 BLE for real-time IMU stabilization, we achieved a balance between high-speed processing and agile physical response. Building a fully autonomous vehicle for the competition has been the most demanding engineering project of our high school career, pushing us far beyond basic robotics and into real-world control theory and computer vision. Our goal this year is to get a better score than last year and improve our knowledge and understanding on robotics. This journal documents the technical evolution, mechanical redesigns, and hard-earned lessons that transformed a failure-prone prototype into a stable, competitive robot.

Last year’s bugs

We also participated in last year’s WRO Future Engineers challenge. A lot of our robots have the same components as last year, but our design, chassis, layout has changed significantly. We also have additional added components, such as the LiDAR and the arduino. Our robot last year had a lot of bugs, we are trying to improve for this year's competition.

- Our 3D design had pegs that would connect the different levels of the robot, but since the pegs were so thin it would easily break.
- We had a LiDAR installed on our robot but never used it, making us unable to complete the obstacle challenge at all.
- Our chassis last year was not suitable for this challenge and would break constantly. Our design was also too high, making it heavy, hard to steer, and losing balance constantly.

Last year's design (2024-2025):



Upgrades

This is the improvements we made from last year to avoid the same mistakes from last year:

- *December 12th, 2025:* To separate and assemble the different layers of our robot, we used the plastic sticks attached already to our chassis and just made holes in our model holding the Raspberry Pi and the power regulator. To hold them in place we used retaining pins that also came with our chassis. This allowed our chassis to be lower to the ground and more stable when turning.
- *November 23rd, 2025:* Our chassis this year has a more stable base and grippier wheels. Better traction allows it to accelerate harder without losing control and maintain a steady speed when turning, making it better for precise tasks like parking and navigating pillars. We also cut part of our chassis so that our steering angle is much larger so it can turn sharper corners or avoid a pillar quickly. It also has much more space than our old chassis, allowing us to keep the battery, servo motor, and drive motor compact and out of the way.
- *March 19th, 2026:* Using new technology like the Arduino Nano provides benefits like how the robot manages timing, signal stability, and real-time responsiveness compared to last year's single board systems. The Arduino Nano operates on bare-metal architecture without an OS. Dedicated hardware timer registers generate 50 HzPWM signals with microsecond accuracy, ensuring smooth servo sweeps and consistent motor speed control without stuttering. A single board system could produce unpredictable micro delays when operating. If high-level vision pipelines, image segmentation, or path-planning code freeze or crash, the Nano runs an independent safety loop.

Issues about last year's code will be provided later on.

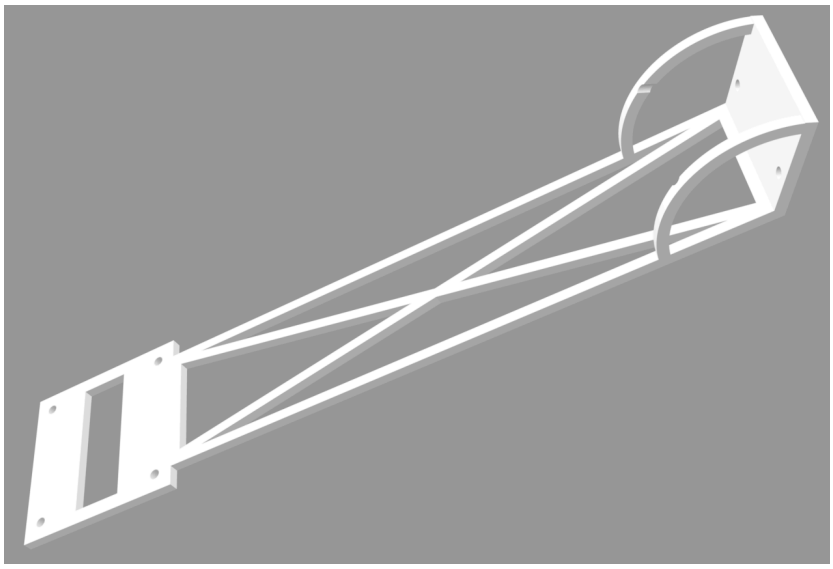
Challenge and System Designs

3D Chassis design:

For our chassis, we needed to 3D print out a platform for our raspberry pi and a ledge to hold up our camera. The following are our process into formulating so.

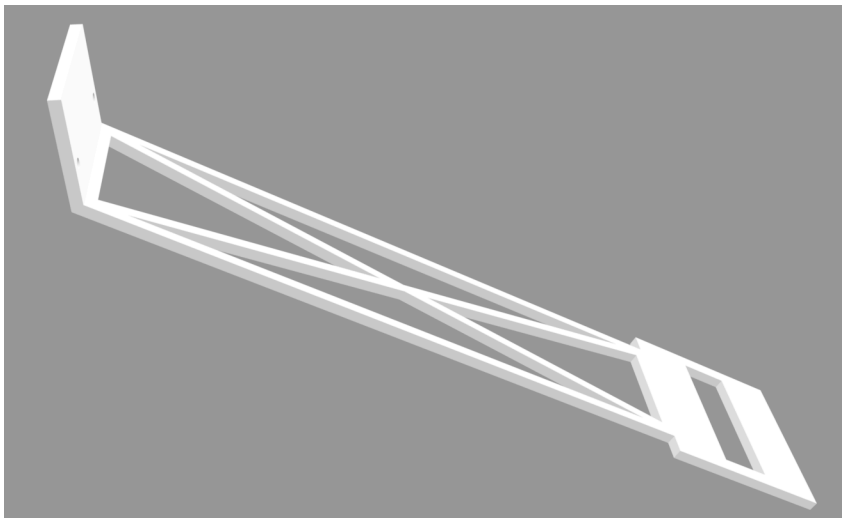
Camera:

November 25th, 2025 Design number 1:



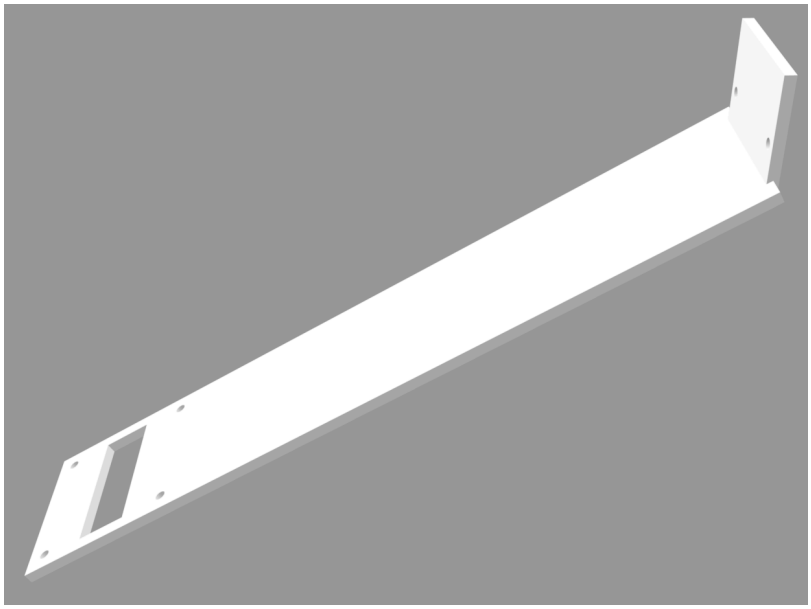
This is our original design for the camera, but it did not print properly because the support pillars were too thin and the round supporters caused some problems. We also realized that the screw holes were too close together and therefore couldn't properly mount our camera into place.

December 5th, 2025 Design number 2:



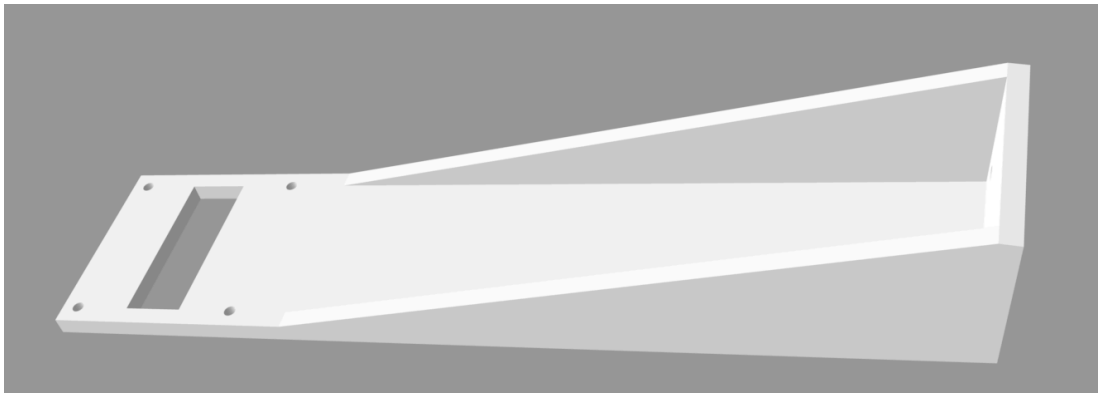
We removed the round supporters because they were too difficult to print. We came up with this new design, and simply just removed the round supporters. We also moved the screw holes slightly out to where it would match up with the screw holes on the camera. The pillars were not able to keep our camera stable and were too flexible, which would severely hinder the camera functions for the open and obstacle challenge.

December 16th 2025 Design number 3:



We replaced the thin pillars into a fully filled in wall. We tried to create a more lightweight design for our chassis, but settled for this instead. Even though this design made our 3D print heavier, it was enough to fully support the camera and display everything properly during the obstacle and open challenge. This 3D print design for our camera ended up not functioning because the angle was low. The robot did not see far enough in front of it so it could prepare to turn smoothly. It also was still too flexible and could bend easily when pushing a force on it.

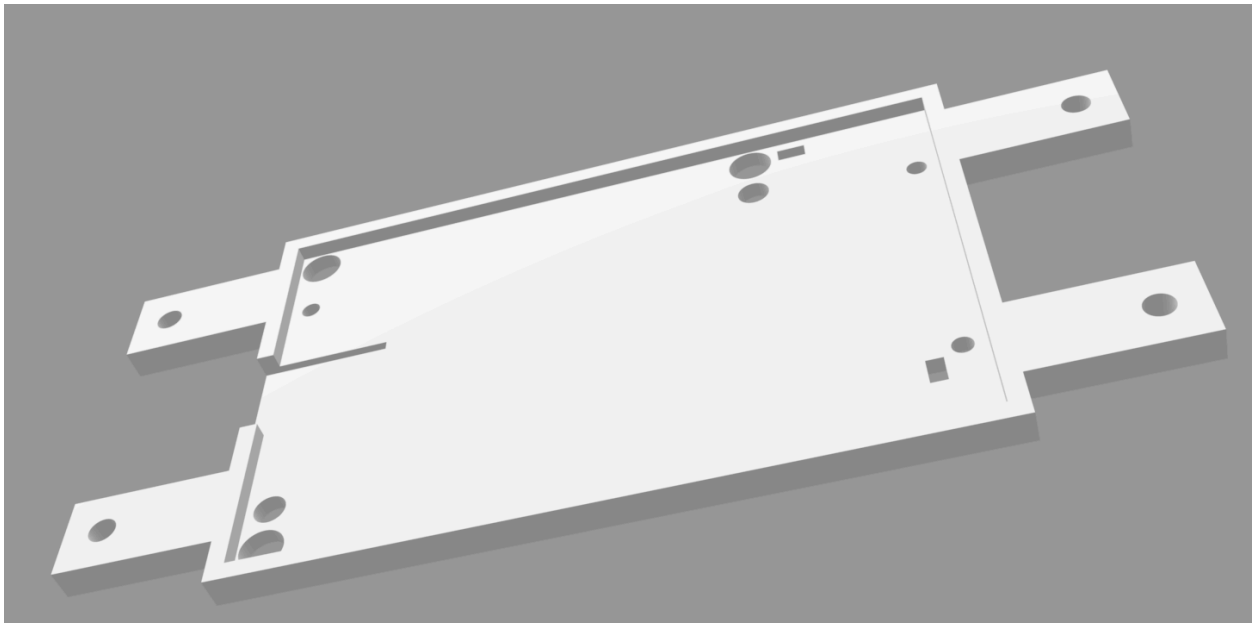
December 19th, 2025 Design number 4 (Final Design):



Our solution was to redesign our print so that the angle was closer to 90º and removed the gaps so it's more stable. This made the chassis print more heavy, but ensured that the camera was very stable when our robot was moving, printed properly, and portrayed everything in front of it for the obstacle and open challenge.

Raspberry pi:

December 5th, 2025 Design number 1 (Final Design not including LiDAR mount):



This is the original and final design for the raspberry pi platform. We measured every screw hole and the length to the chassis supporters so everything would fit into place. We ensured that the platform was high enough so that the motor and battery had enough space underneath. We did have to alter the printing dimensions a couple times, but never changed the original design.

Mechanical Iteration and Mobility

Last Year's Code:

Last year's Open Challenge Code:

March 19th, 2026: We first tried to upload our open challenge code from the previous year. We quickly realized multiple problems from the code and made several changes.

- We needed to upload based on this year's setup. We aren't using the rrc.Board from hiWonder anymore. The obsolete code below was removed:

```
import ros_robot_controller_sdk as rrc

   board = rrc.Board()

   board.set_rgb([[1, 0, 0, 0], [2, 0, 0, 0]])

   def check_node_status():
    command = 'source /home/ubuntu/.zshrc && ros2 topic list'
    result = subprocess.run(
        ['docker', 'exec', '-u', 'ubuntu', '-w', '/home/ubuntu', 'MentorPi',
'/bin/zsh', '-c', command], capture_output=True, text=True)
    res = result.stdout
    return '/ros_robot_controller/button' in res
```

- All serial writes using the format arduino.write(b'@M1600') we made sure didn't have a \n terminator. If the Arduino firmware expects newline-terminated messages, the commands will buffer up and never be parsed.
- We included two modes. Our code only used one single P controller handling both straight sections and corner sections. In the straight section, we wanted a more gentle correction and a higher speed. In the corner, we needed more aggressive steering and likely a slower speed to avoid overshooting. Since a single p-controller can't do both well (being too sluggish on the corners after being tuned for straight and being twitchy after being turned for corners), we needed to change it so our code uses two modes. We made our car drive faster when both ROI's detected black and have a more gentle kp. When only one ROI detects black, we slow down and turn until both ROI's detect black.
- We added a derivative on the error to reduce oscillation.
- Our code had a shared detection flag between the orange and blue detention. We needed to change them to two separate flags (orange_line_detected and blue_line_detected). This is so when an orange line is set to true, the blue direction else branch does not immediately reset to false (and vice versa), helping us count rotations better during the open challenge.
- The "else: line_detected = False" is inside the "if orange_pixels > 100 block", meaning line_detected only resets when pixels are still above threshold. To fix this, it now resets when the line leaves the ROI. Same with blue.

- In our code, we sent too many commands to the arduino. In every frame we sent both @M1600 and @S{pw} (60 commands/sec at 30 fps). To fix this, we send commands only when a value needs to be changed.
- We used computed contours_orange and contours_blue, but we didn't use it, so we removed the findContours calls to save CPU.

Our new and updated open challenge code is provided in the READ ME.

Last Year's Obstacle Challenge Code:

The previous year, we did not complete the coding of our obstacle challenge. This year, we are planning to improve and navigate around the red and green pillars on the corresponding side. We are going to progress into working the obstacle challenge and improving the open challenge for our new design. We are also going to incorporate new technology introduced to us this year, like the Arduino Nano.

This Year's Code:

August 23rd, 2026: In this year's new and updated code, we incorporated several targeted engineering strategies across the open and obstacle challenges to optimize autonomous track navigation, obstacle avoidance, and system safety. For the Open Challenge, stability and reliable cornering are achieved by requiring a multi-frame debouncing check (TRIGGER_FRAMES = 5) before transitioning into corner turn mode, protecting the system from false state changes caused by track glares or shadows. While in wall follow mode, a proportional controller (kp = 0.009) calculates the differential between left and right wall pixel areas to continuously maintain a centered trajectory, while active cornering locks in fixed steering angles until the opposite wall area recovers above 1000 pixels. Lap tracking is similarly stabilized using a 5 second software cooldown timer (cooldown_seconds = 5) across the orange and blue line ROIs to eliminate double-counting, automatically bringing the robot to a complete stop (@M1500) once 3 laps are registered. For the Obstacle Challenge, the vision pipeline assigns top execution priority to pillar detection whenever a contour exceeds 900 pixels, overriding standard wall-following logic to prevent imminent collisions. If both colored markers enter the frame simultaneously, an area arbitration check prioritizes the larger contour to navigate around the closer obstacle first, using a dynamic proportional steering calculation based on the pillar's horizontal centroid offset relative to target positions (x=130 for red and x=550 for green). To handle sudden visual loss during aggressive bypass maneuvers, a temporal fallback counter (pillar_lost_counter > 8) safely reverts the system to wall follow mode if the pillar leaves the camera frame, while all output servo angles are continuously run through a physical clamping function (MIN_SERVO=70, MAX_SERVO=140) to safeguard mechanical linkages against over-extension.

Mobility reasoning:
July 16th, 2026: Throughout the obstacle challenge we changed our kp to 0.009 instead of 0.0075 because this changed value makes the navigation the smoothest in order to complete the challenge in time and with efficiency. The 20% increase forces the servo to react faster when entering sharp corners or quick changes around pillars. It reduces tracking errors, returning back on track faster, especially with the unexpected added obstacles during the competition. It also allows the different modes to have enough time to switch smoothly and with ease.

Mechanical Iteration progression:

At the beginning, the steering assembly utilized stock plastic linkages with a direct servo. Under aggressive proportional corrections in wall follow mode, mechanical play in the stock ball joints caused 2^0–3^0 of steering slope. This slope resulted in subtle front-wheel oscillation flutter on high-speed straightaways, skewing our visual camera alignment and causing false wall-distance readings. We redesigned the front steering linkage, replacing the stock rods with custom 3D-printed rigid tie-rods mounted to an STL bracket, while repositioning the steering servo 5 mm lower on the chassis deck. Additionally, we cut restricting areas off the chassis in order to make space for the steering angle to widen. Shifting weight toward the rear wheels increased rear tire traction during rapid acceleration, reducing wheel slip and stabilizing straight-line camera ROI tracking.

Navigation Modes

Below are the different navigation modes our robot uses and explanations/reasonings to the chosen values and concepts.

Wall follow mode:
The vision pipeline evaluates two mirrored ROI's to monitor the outer black boundaries. By calculating the ratio of detected black pixels in each ROI, the software computes a real-time tracking error. Balancing visual weight across both frames prevents harsh overcorrections, replacing chaotic zigzagging with smooth, linear tracking. Keeping the vehicle dynamically locked to the lane centerline maximizes clearance on both sides, mitigating collision risks and saving vital battery power through reduced steering drag.

Corner turn mode:
The visual imbalance as a state trigger allows the robot to instantly recognize a track corner without needing complex, computationally expensive scene reconstruction. The asymmetry directly dictates the turning trajectory. This removes ambiguity and eliminates latency between corner detection and turn execution. The spatial trigger prevents over-turning once the vehicle aligns with the new straightaway, while the temporal timeout acts as a failsafe to keep the vehicle moving forward if track reflections or wall gaps briefly disrupt visual tracking.

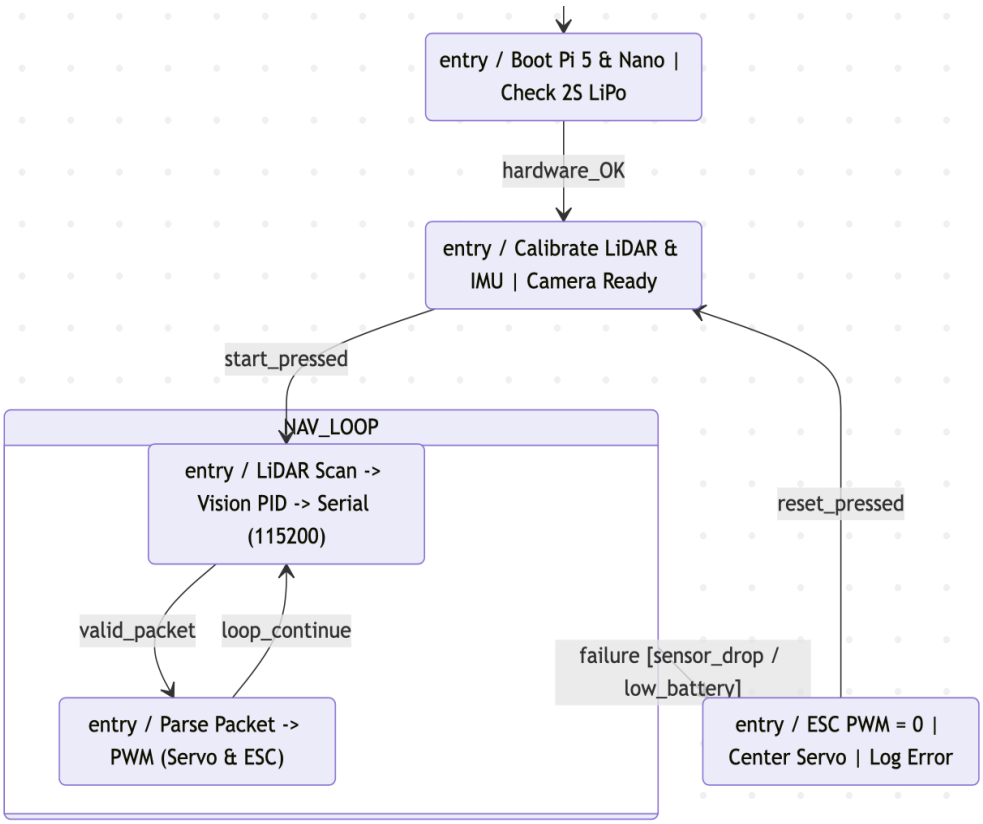
Obstacle avoid mode:
Requiring a minimum contour area of 900 pixels acts as a spatial noise filter. It prevents the system from triggering false-positive obstacle states when distant pillars, wall reflections, or background visual artifacts briefly appear in the camera's field of view. The robot only reacts when a pillar is close enough to present an immediate collision risk. Red pillar values being $X \geq 150, Y \geq 250$ allows the robot to wait for the pillar's bounding box center to reach these threshold coordinates and ensures the robot's rear wheels have physically cleared the right side of the red obstacle before straightening out. Same with the green pillars as well.

Power Analysis

Subsystem interaction:

In order to control our robot, it undergoes subsystem interactions to transfer its perceptions to the motor.

(Camera/LiDAR) —>(Raspberry Pi 5)—>(Serial/UART)—>(Arduino Nano)—>(ESC & Servo)
(Perception) (Path Planning) (Motor Control) (Actuation)



1. **Power distribution & Signals**

The power subsystem provides clean logic power while supplying high current to the drivetrain without inducing voltage sag or brownouts on the Raspberry Pi 5. The Gens Ace 2S LiPo connects directly to the drive motor's ESC. The converter taps off the 2S battery to supply a stable 5V/5A rail directly to the Raspberry Pi 5 (via USB-C or GPIO power pins). The Arduino Nano draws its power directly from one of the Raspberry Pi 5's USB ports, ensuring logic ground is shared across both computing boards.

2. **Perception & Path Planning**

The Pi processes these arrays to compute wall distances on both sides of the vehicle, detecting open corridors, 90◦ track turns, and outer boundary walls. The camera captures color frames on a background thread. OpenCV processes these frames using HSV color masks or light object detection models to spot red and green traffic pillars, estimating their bounding box positions and distance relative to the front bumper. The error passes through the tuned Proportional-Derivative loop (Kp=0.009) to derive the exact target steering angle and target motor velocity.

3. **Inter Communication**

The Arduino Nano's serial buffer reads non-blocking incoming characters until the newline character (`\n`) is reached. A lightweight string parser converts the ascii numbers into integer variables inside the microsecond loop.

4. **Microcontroller Execution**

The servo receives high-frequency timing directly from the Nano's hardware timer register, preventing steering flutter. The target velocity value is converted into standard RC ESC signals (1.0 ms full reverse / neutral brake at 1.5 ms / 2.0 ms full forward).

5. **Automated Failsafe loop**

The Arduino Nano tracks elapsed time since the last valid serial packet using millis(). If no command arrives from the Pi for more than 200 ms (due to a software crash, freeze, or disconnected cable), the Nano automatically resets motor PWM to neutral (SPEED:0). If the Pi detects LiDAR signal drops, battery voltage falling below 7.0V, or an emergency wall proximity threshold (<5 cm), it transmits an immediate override packet (STEER:0,SPEED:0\n), forcing the Arduino to lock steering straight and apply active motor braking.

Performance Calculations

August 16th, 2026: To optimize our 1:24 scale rear-wheel-drive (RWD) platform for the WRO track layout, we selected a 3450 KV brushless motor powered by our 2S LiPo battery (7.4V nominal). At full charge (8.4V peak), the motor reaches a theoretical maximum unloaded speed of approximately 28,980 RPM:

$$RPM_{max} = 3450\ KV \times 8.4\ V = 28{,}980\ RPM$$

Accounting for our drivetrain gear reduction and wheel diameter, this KV rating provides a top speed capable of completing a 3-minute match run well within track requirements while keeping motor temperatures low. Lower KV brushless options lacked the acceleration burst needed to rapidly recover speed after sharp turns or aggressive obstacle bypasses. Conversely, higher KV motors produced excessive wheelspin on our low-friction chassis tires and drew high current spikes, risking voltage brownouts on our logic rails. The 3450 KV motor represents the optimal balance of immediate low-end acceleration out of corners and controllable top-end velocity on straightaways.

Conclusion

Through iterative hardware refinement and several improvements, our 1:24 scale autonomous platform successfully fulfills all technical requirements for the both challenges in the WRO competition. By pairing the high-level computer vision and spatial processing of the Raspberry Pi 5 with the deterministic, real-time actuation of the Arduino Nano, the system achieves stable wall-following, reliable corner transitions, and responsive pillar avoidance. Comprehensive power management, multi-frame vision debouncing, and custom mechanical linkage upgrades ensure high operational reliability across both the Open and Obstacle challenges. Detailed build logs, system schematics, and final high-resolution photographs of the completed robot assembly are available in the project's README on Github.

Repository Link :https://github.com/nicolejin16/Flight/tree/main